

OMNYX Model Selection — Final Decision Report

Purpose: pick the best on-prem AI model for each job in the OMNYX agent system, based on a real test against live Unicharm plant data. Written for decision-making. **Date:** 2026-06-03 · **Supporting data:** [REAL_DATA_MODEL_EVAL.md](#) (local) · [OPENROUTER_MODEL_EVAL.md](#) (cloud big-models).

1. The decision (use these models)

Job	Choose	Backup	Runs on
Planner (decide the action)	gpt-oss:20b	gemma3:27b	20 GB ✓ · 48 GB ✓
Validator (approve / reject)	phi4	qwen2.5:14b	20 GB ✓ · 48 GB ✓
Executor (call the tools)	qwen2.5:14b	mistral-small3.2	20 GB ✓ · 48 GB ✓
NL→SQL (question → database)	gpt-oss:20b + guardrails	mistral-small3.2	20 GB ✓ · 48 GB ✓
RAG (answer from manuals)	gpt-oss:20b	phi4 / mistral	20 GB ✓ · 48 GB ✓
Narration (summarise numbers)	qwen2.5:14b	phi4	20 GB ✓ · 48 GB ✓
Embeddings (search)	nomic-embed-text	mxbai-embed-large	tiny, both ✓

In one line: a small, efficient team — **gpt-oss:20b + phi4 + qwen2.5:14b** — wins. They fit together on the future 48 GB box and each run on today's 20 GB box. **No big/expensive model and no hardware upgrade is needed.**

2. What we tested (plain English)

- Every model was given **real Unicharm plant data** (live telemetry + the real document manuals), not made-up examples.
- We scored **7 jobs**: Planner, Validator, Executor, NL→SQL, RAG, Narration, Embeddings.
- Each answer got **two checks**: (1) a **hard rule** — is it valid (good JSON / runnable SQL / a grounded work order)? — and (2) a **judge AI** ([qwen2.5:32b](#), same for everyone) rating quality **1–5**. **Pass = 4 or more**.
- **Small models (≤32B)** ran on our own 20 GB machine. **Big models (70B/120B)** were tested through the cloud (OpenRouter) — for **quality comparison only** (cloud speed ≠ our speed).
- This was a **quick ranking run** (a few cases per job) — enough to choose, not the final certification. A full-depth re-run is the last step before go-live.

3. Results — who won each job

Scores are average quality (1–5); "pass" = scored 4+.

Job	Winner (score)	Close behind	Avoid (why)
-----	----------------	--------------	-------------

Job	Winner (score)	Close behind	Avoid (why)
Planner	<code>gemma3:27b</code> (4.0) / <code>gpt-oss:20b</code> (3.5)	<code>qwen2.5:14b</code> (3.5)	<code>deepseek-r1</code> (slow 75 s, weak)
Validator	<code>phi4</code> (5.0) — <i>many tie at 5.0</i>	<code>qwen2.5:14b</code> , <code>mistral</code>	<code>deepseek-r1</code> (2.5, 141 s)
Executor	<code>qwen2.5:14b</code> (3.5)	<code>gpt-oss:20b</code> , <code>mistral</code> (3.0)	<code>deepseek-r1</code> (1.5, 434 s/case), <code>qwq</code> (226 s)
NL→SQL	<code>gpt-oss:20b</code> (3.2) — <i>weak field</i>	<code>mistral</code> (2.8)	<code>deepseek-r1</code> & <code>qwq</code> (failed all), even the coder model (2.25)
RAG	<code>gpt-oss:20b</code> (4.4) — <i>most tie ~4.4</i>	<code>phi4</code> , <code>mistral</code>	—
Narration	<code>phi4</code> / <code>qwen2.5:14b</code> (4.5)	<code>mistral</code> , <code>gemma3</code>	<code>deepseek-r1</code> (errored), <code>llama3.1</code> (3.0)
Embeddings	<code>nomic = mxbai</code> (5.0)	—	—

Three things to remember:

- Reasoning models (`deepseek-r1`, `qwq`) are the wrong choice** for structured jobs — they're slow and break JSON/SQL. Good only for narration.
- NL→SQL is weak for everyone** — even the dedicated coder model. This is a **prompt + safety-check problem, not a model-size problem**. It's the #1 thing to engineer.
- `gpt-oss:20b` is the most reliable all-rounder** and fits everywhere — make it the anchor.

4. Do we need a bigger (70B/120B) model? — **No.**

We ran the big cloud models on the same jobs. Bigger did **not** win:

Job	Best small (≤32B)	Best big (70B/120B)	Bigger better?
Planner	<code>gpt-oss:20b</code> 4.0	<code>llama-70b</code> 3.67 · <code>qwen-72b</code> 3.0 · <code>gpt-oss-120b</code> errored	No
Validator	<code>phi4</code> 5.0	<code>llama-70b</code> 5.0	No (already maxed)
Executor	<code>qwen2.5:14b</code> 3.5	<code>qwen-72b</code> 3.33 (slower)	No
RAG	<code>gpt-oss:20b</code> 4.4	<code>llama-70b</code> 4.4	Tie
Narration	<code>phi4/qwen2.5:14b</code> 4.5	<code>llama-70b</code> 4.5	Tie

The 70B models only **tie** at best, run **slower**, and the 120B even **timed out** on planning. **Conclusion: stick with the small team. A bigger model is not worth bigger hardware.**

5. Will it fit our hardware?

We have two machines. Our picks fit **both**.

	Today (test box)	Future (production)
GPU	20 GB	48 GB
RAM	32 GB	48 GB
Can hold	one model at a time	all 3 agents at once

- **Today (20 GB):** only one model loads at a time, so agents take turns (the slow "model swap" you saw). Fine for testing.
- **Future (48 GB):** Planner + Validator + Executor (**13 + 9 + 9 ≈ 31 GB**) all stay loaded together, ~17 GB to spare → no swapping, agents run side by side. **This is the right size.**

6. If you ever want a bigger model — what hardware you'd need

The future box plans **FP8** serving (~1 GB per billion parameters; ~2× the memory of the Q4 we tested). Rough VRAM needed:

Want to run	VRAM needed	Fits 48 GB?
Our 3-agent stack (14–20B), all loaded	~40 GB	☑ yes
One 32B model	~36 GB	☑ yes
One 70B alone (4-bit)	~40 GB	☑ yes, but only one model
One 70B alone (FP8)	~75 GB	✗ need 80 GB
70B + a validator together	~64–80 GB	✗ need 80 GB GPU
120B	~80–130 GB	✗ need 80–96 GB+

Plain answer: ≤32B → the planned 48 GB box is enough. A 70B would need it *to itself* (losing the side-by-side benefit) or an **80 GB** GPU. 120B isn't realistic on-prem. And per section 4, **none of that extra spend buys better results.**

7. Final recommendation

1. **Deploy the small team:** Planner `gpt-oss:20b`, Validator `phi4`, Executor `qwen2.5:14b`, plus `gpt-oss:20b` for NL→SQL/RAG, `qwen2.5:14b` for narration, `nomic-embed-text` for search.
2. **Keep the planned 48 GB box** — no upgrade needed.
3. **Invest engineering effort in the NL→SQL safety/retry layer** — that's the real risk, not the model choice.
4. **Do a full-depth re-run** (more cases per job) for final sign-off, plus the Round-3 vLLM/FP8 latency test on the 48 GB hardware.
5. This **confirms the earlier synthetic pick** (gpt-oss:20b Planner + phi4 Validator) on real data — safe to lock in.

Housekeeping: rotate the OpenRouter API key (it was pasted in chat); it's stored gitignored at `model-eval/.env.local`.